



YNOV CAMPUS · BACHELOR 1 INFORMATIQUE

Découverte des Langages compilés

Du code source aux instructions machine — C & C++

DURÉE 20h sur 5 jours (4h/jour)

ECTS 2 crédits · Module : Les bases du développement

PÉRIODE 11 mai 2026 → 30 juin 2026

ÉVALUATION Projet pratique en C/C++

FORMATEUR

Ayoub Benhamdi

Data Scientist & AI Engineer
Montpellier, France

JOUR

01

sur 05

DATE DE SESSION

lundi 11 mai 2026

Durée : 4h en présentiel

Introduction & premier programme

Programme de la journée

Accueil, présentation du module et des objectifs

Compilé vs interprété : concepts et écosystème C/C++

Mise en place de l'environnement de développement

Atelier — votre premier programme C compilé

Bienvenue dans le monde compilé

Module "Découverte des langages compilés"

*« Un programme bien compilé
est déjà à moitié exécuté. »*

Pendant 5 jours, vous allez découvrir comment votre code source est transformé en instructions machine, comprendre la performance offert par C/C++, et écrire vos premiers programmes compilés.

20h

de cours

5

journées

C/C++

langages

Les objectifs du module

À l'issue des 5 jours, vous serez capable de...



Distinguer compilation et interprétation

Comprendre les différences fondamentales entre un programme compilé et un programme interprété, et savoir quand utiliser chaque approche.



Expliquer le processus de compilation

Maîtriser les 4 étapes : préprocesseur, compilation, assemblage, édition de liens.



Programmer en C et C++

Écrire, compiler et exécuter des applications simples : variables, structures de contrôle, fonctions, classes.



Identifier et corriger des erreurs

Lire un message du compilateur, distinguer erreurs et warnings, déboguer un programme.



Comprendre la gestion mémoire

Stack, heap, allocation dynamique, fuites mémoire — les bases pour des programmes robustes.

Programme du module

5 journées pour maîtriser les bases

01

JOUR

Lun. 11/05

**Introduction &
premier programme**

Compilé vs interprété,
écosystème C/C++, Hello
World

02

JOUR

Lun. 18/05

**Le processus de
compilation**

Préprocesseur, compilation,
assemblage, linker — bases
du C

03

JOUR

Lun. 01/06

**Structures, fonctions,
pointeurs**

Contrôle de flux, fonctions,
tableaux, premières
adresses mémoire

04

JOUR

Lun. 08/06

**Gestion mémoire &
C++**

Stack, heap, malloc/free,
classes et POO

05

JOUR

Lun. 15/06

**Compilation vs
runtime & projet**

Optimisations, debug,
bonnes pratiques, projet
final

Pourquoi étudier les langages compilés ?

Une compétence fondatrice du métier d'informaticien

Performance. Les programmes compilés s'exécutent à la vitesse native du processeur.

Contrôle bas-niveau. Vous gérez la mémoire, les types, l'allocation — vital pour comprendre comment marche un ordinateur.

Universalité historique. C est le langage du système d'exploitation : Linux, Windows, macOS, kernels, drivers.

Fondations des autres langages. Python, Java, JavaScript, Ruby — leurs interpréteurs sont eux-mêmes écrits en C/C++.

Embarqué et IoT. Microcontrôleurs, voitures, satellites, électroménager : C est partout où la ressource est limitée.

Jeu vidéo & moteurs 3D. Unreal Engine, moteurs physiques, calculs graphiques temps-réel — du C++ partout.

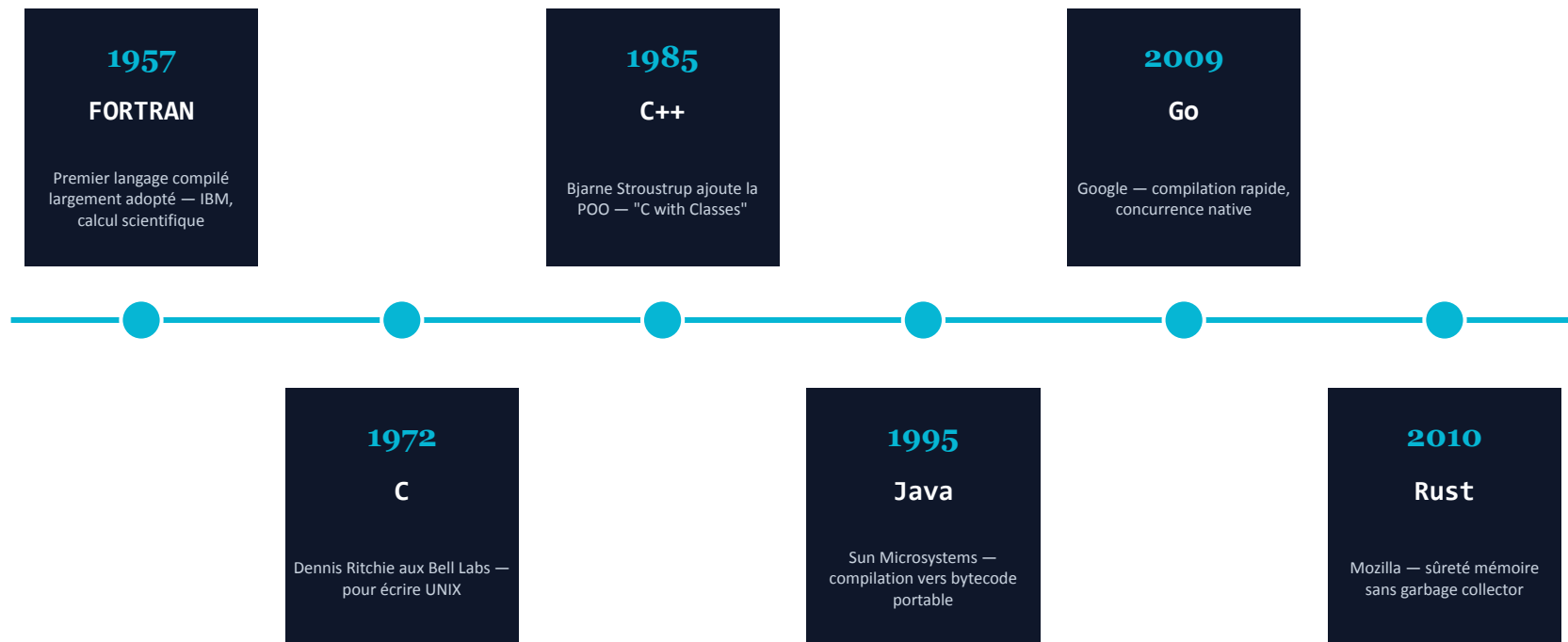
LE SAVIEZ-VOUS ?

Le noyau Linux compte environ 27 millions de lignes de code, écrites majoritairement en C.

La NASA utilise C pour le code embarqué de ses sondes spatiales — un C particulièrement strict et vérifié.

Une brève histoire des langages compilés

Des cartes perforées aux compilateurs modernes



Du code source à l'exécution

Que se passe-t-il quand on "lance" un programme ?

Un ordinateur ne comprend que des 0 et des 1.

Les processeurs exécutent uniquement du code binaire — des suites d'instructions machine spécifiques à leur architecture (x86, ARM...). Le code que vous écrivez en C, Python ou JavaScript doit donc être traduit. Cette traduction peut se faire avant l'exécution (compilation) ou pendant (interprétation).

Exemples concrets

Code source

Texte lisible par un humain que vous écrivez dans votre éditeur (.c, .cpp, .py...).

Code machine

Suite d'instructions binaires propres au processeur (x86, ARM, RISC-V...).

Le pont

Compilateur ou interpréteur — le traducteur qui fait le lien.

Langage compilé : la définition

Tout traduire avant d'exécuter

Étape 1 — Vous écrivez le code source Dans un fichier texte (ex. main.c).

Étape 2 — Le compilateur traduit l'intégralité du code Avant toute exécution, en code machine binaire.

Étape 3 — Un fichier exécutable est produit main.exe sous Windows, ./main sous Linux/Mac.

Étape 4 — Le programme s'exécute directement Le processeur lit les instructions binaires sans intermédiaire.

Conséquence : performance maximale Mais code lié à un OS / une architecture (recompilation nécessaire pour changer).

EXEMPLES

C, C++, Rust, Go, Fortran, Pascal, Ada, Swift (compilé), Kotlin Native, Objective-C.

Leurs exécutables sont autonomes : pas besoin d'installer un "interpréteur" pour les lancer.

Langage interprété : la définition

Traduire ligne par ligne, en direct

Pas de phase de compilation préalable Le code source est lu directement à l'exécution.

Un "interpréteur" lit chaque instruction Et la traduit en temps réel pour le processeur.

Pas d'exécutable indépendant Vous distribuez le code source — l'utilisateur doit avoir l'interpréteur.

Avantage : portabilité immédiate Le même fichier .py fonctionne sur Windows, Mac et Linux sans modification.

Inconvénient : exécution plus lente Le travail de traduction se fait à chaque exécution.

EXEMPLES

Python, Ruby, PHP, JavaScript (Node.js), Bash, Lua, R, Perl.

Tous nécessitent leur runtime installé sur la machine cible.

Compilé vs Interprété — Comparaison directe

Deux philosophies, deux compromis différents



Compilé (C, C++, Rust)

- Vitesse d'exécution maximale
- Optimisations agressives possibles
- Exécutable autonome (pas de runtime requis)
- Erreurs détectées avant exécution
- Code lié à un OS / une architecture
- Cycle de développement plus long (compiler à chaque modification)



Interprété (Python, JavaScript)

- Cycle de développement très court
- Portable : un seul fichier sur tout OS
- Idéal pour le prototypage rapide
- Erreurs souvent détectées à l'exécution
- Performances plus faibles
- Nécessite un interpréteur sur la machine cible

Où utilise-t-on C et C++ aujourd'hui ?

Des domaines où la performance fait la différence

Systemes d'exploitation

Noyaux Linux, Windows, macOS, BSD
— drivers et code système.

Embarqué & IoT

Voitures, drones, IRM, satellites,
électroménager connecté.

Moteurs de jeu

Unreal Engine, Unity (cœur), CryEngine
— calculs temps réel.

Calcul scientifique

Simulations météo, physique des
particules, biologie.

Bases de données

PostgreSQL, MySQL, Redis, SQLite —
performance critique.

Bibliothèques de Python

NumPy, Pandas, TensorFlow — leur
cœur est en C/C++.

Navigateurs web

Chromium, Firefox — moteurs
JavaScript et de rendu.

Trading haute fréquence

Latence en microsecondes — C++ règne
sur les marchés.

L'écosystème C / C++

Deux langages frères, deux philosophies



1972 · Dennis Ritchie · Bell Labs

- Procédural
- Proche du matériel
- Léger, rapide à compiler
- Standard ANSI C, C99, C11, C23
- Le langage des systèmes



1985 · Bjarne Stroustrup

- Multi-paradigme (procédural, OO, générique)
- Ajoute classes, templates, exceptions
- Bibliothèque standard riche (STL)
- Standards C++11/14/17/20/23
- Langage des grandes applications

Votre environnement de développement

Les outils dont vous aurez besoin



Un éditeur de code ou un IDE

VS Code (gratuit, multi-plateformes) — recommandé. Alternatives : CLion, Code::Blocks, Visual Studio.



Un compilateur C/C++

GCC (GNU Compiler Collection) sur Linux/Mac, MinGW ou MSVC sous Windows. Clang est aussi un excellent choix.



Un terminal

Bash sur Linux/Mac, PowerShell ou Git Bash sous Windows. C'est là que vous lancerez vos compilations.



Un debugger

GDB est intégré à GCC — indispensable pour comprendre ce qui se passe à l'exécution (Jour 5).

Installation rapide de GCC

Trois plateformes, trois commandes

Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install build-essential
```

build-essential apporte gcc, g++, make et les en-têtes système.

macOS

```
xcode-select --install
```

Installe les Command Line Tools : gcc, clang, make et git.

Windows

```
winget install -e --id MSYS2.MSYS2  
# puis dans MSYS2 :  
pacman -S mingw-w64-x86_64-gcc
```

Alternative : utiliser WSL2 pour avoir un environnement Linux complet.

Vérifier l'installation :

```
gcc --version
```

Votre tout premier programme

Hello World — la tradition incontournable

```
main.c — gcc main.c -o main

// hello.c — votre premier programme C
#include <stdio.h>

int main(void) {
    printf("Bonjour le monde compile !\n");
    return 0;
}
```

- `#include <stdio.h>` — on importe la bibliothèque standard d'entrées/sorties.
- `int main(void)` — point d'entrée du programme : la fonction qui sera exécutée en premier.
- `printf` — affiche du texte sur la sortie standard. Le `\n` produit un saut de ligne.
- `return 0` — code de sortie : 0 signifie « tout s'est bien passé ».

Compiler et exécuter pas à pas

De *hello.c* à *hello.exe* en 2 commandes

```
Terminal — bash

$ gcc hello.c -o hello
$ ./hello
Bonjour le monde compile !
$ echo $?
0
```

Décortiquons la commande :

gcc

Le compilateur GNU

hello.c

Le fichier source à compiler

-o

Option "output" — précise le nom du fichier produit

hello

Le nom de l'exécutable produit

Atelier 1 — Mes premiers programmes

Atelier pratique



MISSION

Installez votre environnement, compilez votre premier programme C et expérimentez avec l'affichage et les codes de sortie.

Étapes à réaliser

1. Vérifier que GCC est correctement installé : `gcc --version`
2. Créer un fichier `hello.c`, taper le code Hello World et le compiler.
3. Modifier le programme pour afficher votre prénom et votre âge sur 2 lignes.
4. Créer un programme qui retourne un code de sortie différent de 0 et le vérifier avec `echo $?`.
5. Tenter une erreur volontaire (oublier le ; ou les guillemets) et lire le message d'erreur de GCC.



ASTUCE ·

Sauvegardez vos fichiers dans un dossier dédié au cours. Nous y reviendrons tous les jours.

Bilan du Jour 1

Ce que vous savez maintenant faire



ACQUIS DE LA JOURNÉE

- Distinguer un langage compilé d'un langage interprété
- Citer les grands cas d'usage de C et de C++
- Installer et vérifier GCC sur votre machine
- Écrire, compiler et exécuter un programme C minimal
- Lire un message d'erreur GCC simple
- Comprendre ce que font printf, return et le code de sortie



PROCHAINE ÉTAPE

Demain, nous plongerons dans la mécanique interne de la compilation.

Vous découvrirez les 4 étapes que GCC exécute en réalité quand vous tapez "gcc hello.c", et nous commencerons à programmer pour de vrai en C : variables, types, opérateurs, lecture clavier.

Apportez votre code du jour — nous l'enrichirons.